MEMORIA DE LA SEGUNDA ENTREGA INDIVIDUAL

Jon Gete Muñoz

DESARROLLO AVANZADO DE SOFTWARE

ÍNDICE:

- **Introducción y Enlace al Repositorio**
- **Arquitectura del Sistema y Base de Datos**
- **Estructura del Proyecto y Clases**
- **Elementos de Valoración Avanzados**
- **Manual de Usuario**
- **Dificultades Encontradas**
- **Bibliografía y Fuentes Utilizadas**

1. Introducción y Repositorio de Código

EcoRuta es una aplicación móvil nativa desarrollada para el sistema operativo Android, cuyo objetivo principal es fomentar la participación ciudadana en el cuidado del medio ambiente. La plataforma permite a los usuarios geolocalizar, reportar y gestionar incidencias medioambientales (como vertidos de basura o zonas a limpiar) en tiempo real.

Para su desarrollo se ha optado por lo siguiente: el cliente móvil ha sido programado en Java utilizando los componentes arquitectónicos modernos de Android (como `WorkManager` y `ActivityResultLauncherS`), mientras que la cartografía se ha implementado mediante la librería de código abierto OSMDroid. El backend se apoya en un servidor Apache con scripts en PHP que exponen una API RESTful, conectada a una base de datos relacional MySQL.

El código fuente completo del proyecto, incluyendo tanto la aplicación Android como los servicios web (ficheros `.php`) y el script de la base de datos, se encuentra alojado y versionado en el siguiente repositorio:

- **Enlace al Repositorio Git:** https://github.com/JG1t1/Ecoruta-DAS.git

2. Arquitectura del Sistema y Base de Datos

La aplicación sigue un modelo de arquitectura Cliente-Servidor :

1. Cliente (Android): Se encarga de la interfaz de usuario (UI), la recolección de datos mediante sensores de hardware (GPS y Cámara) y la interacción del usuario. Para no bloquear el hilo principal (Main Thread), todas las comunicaciones con el servidor se delegan a tareas en segundo plano mediante la API `WorkManager`.

2. Servidor y API REST (PHP): Actúa como intermediario. Recibe peticiones HTTP (GET y POST) con cargas útiles en formato JSON, procesa las imágenes codificadas en Base64 para guardarlas en el sistema de archivos local (`/uploads`), y ejecuta las sentencias preparadas en la base de datos.

3. Base de Datos (MySQL): Almacena la información de forma persistente.

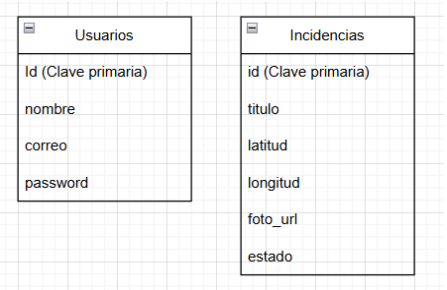


Figura 1: Diagrama de Arquitectura Cliente-Servidor del sistema EcoRuta.

Esquema de la Base de Datos

El modelo de datos se ha diseñado buscando la eficiencia y la normalización, componiéndose de dos tablas principales:

- Tabla `usuarios`: Gestiona el control de acceso. Contiene los campos `id` (Primary Key, Auto-incremental), `nombre`, `correo` y `password`.

- Tabla `incidencias`: Almacena los reportes geolocalizados. Contiene los campos `id` (Primary Key), `titulo` (VARCHAR), `latitud` y `longitud` (DOUBLE para precisión GPS), `foto_url` (Ruta física de la imagen) y `estado` (VARCHAR para gestionar el ciclo de vida del reporte: 'pendiente' o 'resuelta').

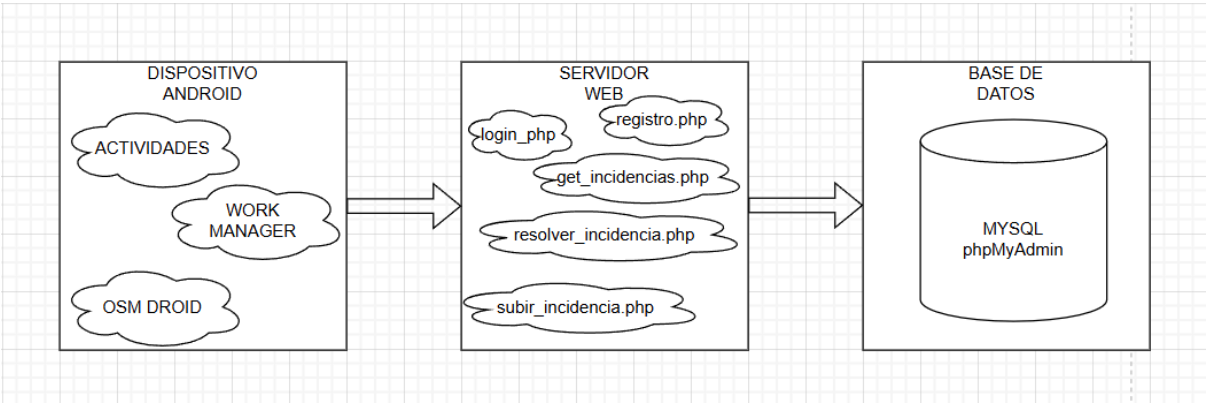


Figura 2: Modelo Entidad-Relación de la base de datos ecoruta.

3. Estructura del Proyecto y Clases

El código fuente de Android se ha estructurado siguiendo el principio de Responsabilidad Única, separando la lógica de la interfaz visual de las operaciones de red. Las clases principales desarrolladas son:

Controladores de Interfaz (Activities)

- `MainActivity` y `LoginActivity`: Gestionan el flujo de registro y autenticación del usuario. Utilizan `HttpURLConnection` para validar las credenciales contra el servidor de forma segura.
- `MapaActivity`: Actividad principal de la aplicación. Integra el `MapView` de OSMDroid, gestiona el ciclo de vida del GPS (`MyLocationNewOverlay`) y contiene la lógica de renderizado y filtrado espacial (búsqueda por proximidad a 10km) de las chinchetas (Markers).
- `ReporteActivity`: Interfaz para la creación de nuevas incidencias. Implementa el hardware de la cámara y el acceso a la galería externa.

Componentes Visuales Personalizados

- `BurbujaPersonalizada`: Hereda de `InfoWindow` de OSMDroid. Permite inyectar un diseño XML personalizado (`MaterialCardView`) al tocar una chincheta en el mapa, integrando la librería externa Glide para la carga de las imágenes desde el servidor y lógica para resolver incidencias localmente.

Gestores de Trabajo Asíncrono (Workers)

- `ReporteWorker`: Empaqueta los datos y la imagen en Base64, enviándolos por POST al servidor garantizando la entrega incluso si la app se minimiza.
- `DescargarIncidenciasWorker`: Realiza peticiones GET periódicas para descargar el JSON con el estado actual del mapa.
- `ResolverWorker`: Comunica al servidor los cambios de estado de una incidencia (de pendiente a resuelta).

Servicios y Tareas Programadas (Características Avanzadas)

- `PatrullaService`: Implementa un *Foreground Service* de tipo *location* que mantiene la aplicación ejecutándose de forma prioritaria, mostrando una notificación persistente al usuario.
- `AlarmaReceiver`: Extiende de `BroadcastReceiver`. Escucha los eventos del `AlarmManager` para lanzar notificaciones locales programadas en días específicos.
- `EcoWidget`: Extiende de `AppWidgetProvider`. Genera una vista remota (`RemoteViews`) incrustada en el escritorio del sistema operativo para acceso rápido.

4. Elementos de Valoración Avanzados

Para dotar a la aplicación de un nivel de complejidad técnica y enriquecer la experiencia de usuario, se han integrado de forma nativa cuatro elementos avanzados del framework de Android, cumpliendo con los requisitos del proyecto:

1. Tareas Programadas (AlarmManager): Se ha implementado un sistema de recordatorios utilizando la clase `AlarmManager`. Mediante un `BroadcastReceiver` (`AlarmaReceiver.java`), el sistema programa respetando los ciclos de ahorro de energía (Doze Mode) una notificación local que se dispara cada viernes a las 18:00, invitando al usuario a realizar tareas de limpieza el fin de semana.

2. Servicios en Primer Plano (Foreground Service): A través de `PatrullaService.java`, se ha creado el "Modo Patrulla". Se trata de un servicio que se mantiene en ejecución máxima prioridad (`START_STICKY`) vinculado a una notificación persistente. Se ha adaptado el código a las políticas de privacidad de Android 14 (API 34), especificando el atributo `foregroundServiceType="location"`.

3. Content Providers (Lector de Galería): Para ofrecer flexibilidad, el usuario no está obligado a tomar la foto en tiempo real. Mediante un `ActivityResultLauncher` y la interfaz `ContentResolver`, la aplicación se comunica de forma segura con el *Content Provider* del sistema operativo (MediaStore) para abrir la galería externa, extraer la URI del archivo seleccionado y decodificarlo a un mapa de bits (`Bitmap`) en memoria.

4. Widget de Escritorio (AppWidgetProvider): Se ha desarrollado el componente `EcoWidget.java`, el cual provee una vista remota (`RemoteViews`) que el usuario puede anclar en su pantalla de inicio (Launcher). A través de un `PendingIntent` con el flag de inmutabilidad (`FLAG_IMMUTABLE`), el widget actúa como un atajo directo que lanza la aplicación principal.

5. Manual de Usuario

El diseño de la interfaz de EcoRuta garantiza una curva de aprendizaje mínima para el usuario final.

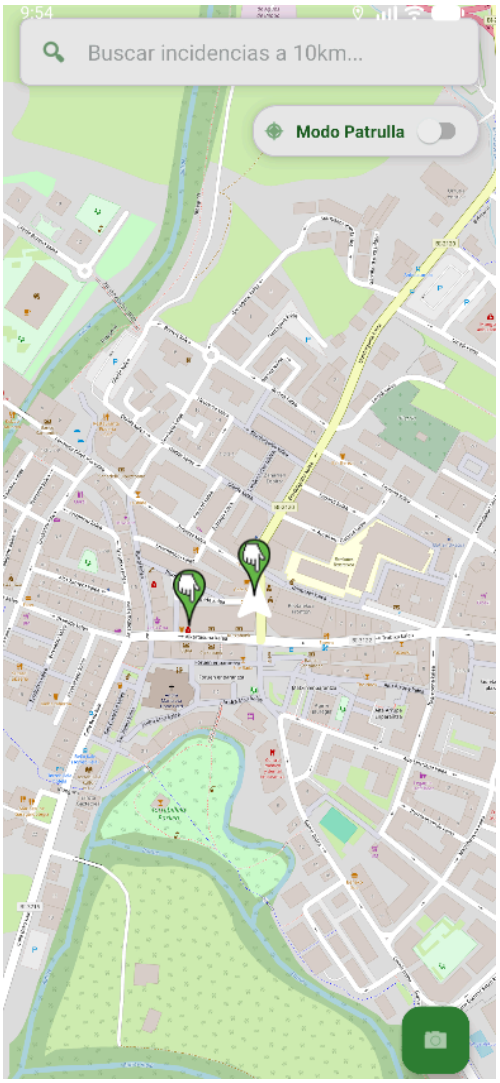
5.1. Autenticación y Acceso

Al iniciar la aplicación, el sistema requiere validación. El usuario debe introducir sus credenciales. Si es nuevo, puede utilizar el botón de registro.

- Usuario de prueba: luis10@gmail.com
- Contraseña:1234

5.2. Navegación y Búsqueda Espacial

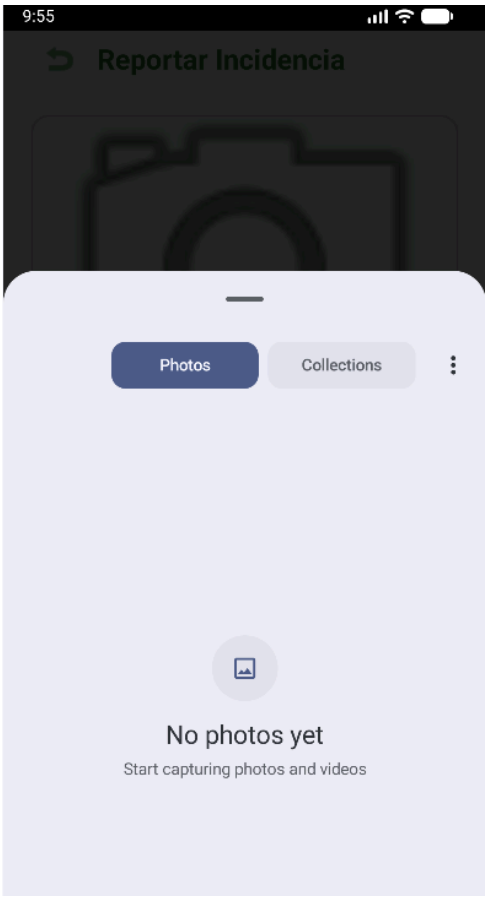
Una vez autenticado, el usuario accede a la pantalla principal. El mapa centrará automáticamente su posición actual utilizando el GPS del dispositivo. En la parte superior, dispone de una Barra de Búsqueda Dinámica que filtra en tiempo real las incidencias mostradas en el mapa que se encuentren en un radio máximo de 10 kilómetros y coincidan con el texto introducido.



5.3. Reporte de Incidencias

Pulsando el botón flotante inferior derecho, se accede al formulario de reporte.

1. El usuario debe proporcionar una imagen tocando el recuadro central (que activará la cámara del dispositivo) o seleccionando el botón inferior para abrir la galería fotográfica del sistema.
2. Tras añadir un título descriptivo, el botón "Enviar" subirá la incidencia al servidor en segundo plano.

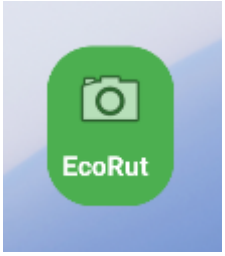


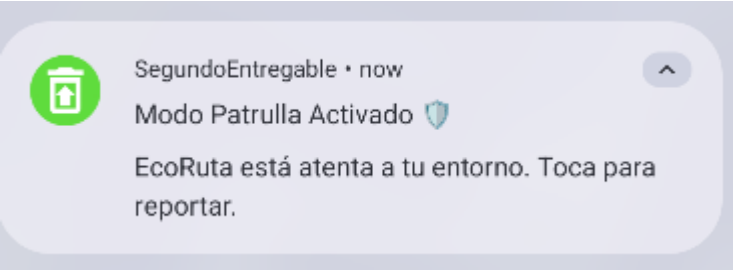
5.4. Resolución de Incidencias (Limpieza)

Al tocar cualquier chincheta en el mapa, se desplegará una tarjeta de información (*InfoWindow*) que muestra la fotografía descargada del servidor y un botón verde de "Limpiado (Resolver)". Al pulsarlo, la incidencia se marcará como resuelta en la base de datos y desaparecerá del mapa para evitar la saturación visual.

5.5. Uso de Herramientas del Sistema (Patrulla y Widget)

El usuario dispone de un interruptor en el mapa para activar el **Modo Patrulla**. Al hacerlo (tras conceder los permisos pertinentes), se generará una notificación fija en la barra del sistema. Además, el usuario puede añadir el widget verde de "EcoRuta" desde los ajustes de su pantalla de inicio para un acceso rápido





6. Dificultades Encontradas

Durante el desarrollo de esta aplicación de arquitectura distribuida, se han superado diversos retos técnicos:

1. **Gestión del Ciclo de Vida de Actividades (Rotación de pantalla):** Al invocar la cámara, la rotación accidental del dispositivo provocaba la destrucción y recreación de la `ReporteActivity`, perdiendo la captura fotográfica alojada en la memoria RAM. Se resolvió bloqueando la orientación a *portrait* en el `AndroidManifest.xml`, garantizando la persistencia visual.
2. **Políticas de Seguridad en Android 13 y 14 (Notificaciones y Servicios):** Las nuevas restricciones de Google requirieron refactorizar la lógica del servicio en primer plano. Fue necesario implementar peticiones dinámicas de permisos en tiempo de ejecución (`POST_NOTIFICATIONS`) y definir tipologías estrictas para los servicios (`FOREGROUND_SERVICE_LOCATION`), evitando así excepciones de seguridad tipo `SecurityException`.
3. **Carga Asíncrona de Imágenes en Mapas:** Mostrar imágenes procedentes de URLs externas directamente en un mapa de OSMDroid bloqueaba el hilo de la interfaz de usuario. La solución técnica aplicada fue integrar la librería **Glide**, que gestiona la descarga en hilos paralelos y almacena las imágenes en caché de forma eficiente dentro de la clase personalizada `BurbujaPersonalizada`.
4. **Saturación del Servidor al Filtrar:** Inicialmente, el buscador realizaba peticiones a la base de datos por cada pulsación de tecla. Para optimizar el rendimiento y reducir la latencia, se modificó la arquitectura: el cliente ahora descarga el JSON completo una sola vez al inicio y aplica los algoritmos de filtrado espacial (cálculo de distancia a 10km) localmente en el propio dispositivo Android.

7. Fuentes Utilizadas

Para la resolución de problemas técnicos y la correcta implementación de las librerías, se ha consultado la siguiente documentación oficial:

- **Documentación Oficial de Android Developers:** Consultada para la implementación de `WorkManager`, `ActivityResultContracts` (Cámara y Content Providers), `AlarmManager` y Servicios en Primer Plano. (https://developer.android.com/docs)

- **OSMDroid (OpenStreetMap para Android):** Documentación del repositorio de GitHub para la inicialización del mapa, gestión de capas (`Overlays`), cálculo de distancias y creación de marcadores personalizados (`InfoWindow`).

- **Librería Glide:** Implementación recomendada por Google para el manejo optimizado de imágenes en Android.

- **Inteligencia Artificial :** Ha sido de ayuda al encontrarme con problemas a la hora de desarrollar la aplicación y a la hora de hacer la documentación.(Gemini)